

Human Activity Recognition using Wireless Signals and Low-Cost Embedded Devices

Thuan V. A. Tong^{*}, Binh Bui-Thanh[†], Phuoc Nguyen T. H.[‡]

University of Information Technology, Ho Chi Minh City, Vietnam

Vietnam National University Ho Chi Minh City, Vietnam

^{*}21522652@gm.uit.edu.vn, [†]binhbt@uit.edu.vn

[‡]phuocnth@uit.edu.vn (corresponding)

Abstract—Developing affordable and privacy-aware smart home applications remains a challenge. Existing camera-based and sensor-based solutions often raise concerns about cost and user privacy. This paper proposes a low-cost WiFi-based sensing system utilizing ESP32 microcontrollers and Jetson Nano edge devices for human activity recognition (HAR) within smart homes. The system captures Channel State Information (CSI) data using an ESP32-based WiFi transmitter and receiver, eliminating the need for expensive hardware while ensuring privacy preservation. A mix of conventional machine learning and deep learning models are evaluated on the obtained dataset, achieving an accuracy of up to 95.57%. The best-performing model is then deployed on the Jetson Nano edge device for efficient activity classification and high throughput. Furthermore, the system seamlessly integrates with existing communication protocols such as Message Queuing Telemetry Transport (MQTT), and provides a user-friendly interface for visualization. We demonstrate the effectiveness of our approach by achieving accurate HAR using a single transmitter-receiver pair, highlighting its practicality and scalability potential. This work aims to further exploration of WiFi sensing technology for smart home applications by enabling cost-effective and privacy-preserving HAR.

Index Terms—WiFi Sensing, Channel State Information, Machine Learning, Human Activity Recognition, Edge Inference.

I. INTRODUCTION

As the demand for convenience, security, and personalized experiences within our living spaces grows, smart home technologies are rapidly transforming the way we interact with our surroundings. Smart home applications using the combination of Internet of Things (IoT) devices and edge computing technologies for machine learning tasks have been the focus of many researches in recent years [1]. Among those applications, it can be seen that there is a focus on the technical ability to precisely track and recognize human actions. A large reason for this is due to its numerous potentials in healthcare, fitness monitoring as well as security at home. Regarding that, several solutions through usages of *camera-based* equipment or *wearable* sensors have been put into practice, a

majority of which yield positive results including high accuracy and low latency [2].

However, taking *user privacy* and *affordability* into consideration, the aforementioned solutions may not meet these requirements. To be more specific, using cameras to monitor people or wearing sensors on their bodies can be viewed as an act of privacy intrusion. Moreover, the cost of both solutions can be expensive to set up and operate. Consequently, an alternative approach that offers both “device-free” monitoring and cost-effective deployment is the use of WiFi signals captured by embedded devices. The widely used terminology for this approach is “WiFi-based human sensing” [3], [4], [5].

WiFi-based human sensing operates on the capture of subtle changes in Channel State Information (CSI) induced by human motion, from which it can extract meaningful information to be used for making predictions. The main idea is to take advantage of the variations of the wireless signal propagated under multi-path fading conditions. This technology benefits from the pervasive nature of WiFi, making it a readily accessible solution for various applications. Nevertheless, collecting CSI data in WiFi signals can be labor-intensive due to complex calibration procedures, lengthy data capture times, and potential manual annotation needs. As a result, the availability of public CSI datasets is limited, which makes it difficult to conduct research in this field.

From a hardware perspective, many existing WiFi sensing systems require rather expensive and intricate hardware such as the Intel 5300 Network Interface Card (NIC) with the Linux 802.11n CSI Tool [6] or the Atheros line of NICs with the Atheros CSI Tool [7]. Aside from that, in some studies, Universal Software Radio Peripheral (USRP) devices have been applied for WiFi sensing purposes due to their ease of configuration and ability to achieve substantially high sampling rates [8]. However, the cost of these devices is even higher. Thus, a restriction is shown in terms of practicality and scalability for these choices of hardware. As a solution

to the pricing problem, Steven M. Hernandez proposed the ESP32 CSI toolkit [4] which utilizes the low-cost ESP32 microcontrollers for CSI collection. While this approach shows promise, further research is crucial to realize its potential in terms of practical application.

In this paper, we aim to advance the field of embedded systems and enrich this field of WiFi sensing by demonstrating the real-world potential of the ESP32 CSI Toolkit for WiFi sensing and contributing a public CSI dataset specifically collected using ESP32.

This paper presents a low-cost Wi-Fi sensing system built on affordable ESP32 microcontrollers to recognize human activities. The system employs a data collection and processing pipeline combined with a machine learning model for activity classification running on the powerful Jetson Nano embedded platform. To facilitate efficient and scalable data exchange, we implement a central communication hub. This hub allows seamless interaction between the internal devices involved in the HAR process and the external devices. As for the external side, a mobile application is included for real-time visualization. This centralized architecture ensures efficient data flow, minimizes latency, and enables easy integration of additional devices in the future.

With the core IoT devices forming a wireless embedded network, a central server for data communication, and a front-end layer for user interactivity, this work's relevance stems from its core contributions, as follows:

- First, we further demonstrated the capability of the ESP32 microcontrollers as a platform for WiFi sensing.
- Second, we collected a CSI dataset for human activity and made it publicly available along with the source code following the link <https://github.com/thu4n/ESP32-WiFi-Sensing>.
- Third, we examined the performance of multiple machine learning and deep learning models to determine their feasibility for edge deployment.
- Finally, we established a simple end-to-end WiFi sensing system for human activity recognition and evaluated the overall operation of the system.

The rest of this paper is organized as follows. In Section II, we provide a background on WiFi sensing and discuss its related work. In Section III, we go through the details of the proposed system. We evaluate the system in Section IV and lastly, we conclude our work and findings in Section V.

II. PRELIMINARIES

A. WiFi Sensing's Background

The main principle behind WiFi sensing is utilizing the CSI signal in the surrounding environment. CSI is a

signal metric captured in orthogonal frequency-division multiplexing (OFDM) transmission system which describes amplitude and phase variations across multiple subcarrier frequencies as wireless signals propagate from a transmitter to a receiver. In the frequency domain, the OFDM system is modeled as:

$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n} \quad (1)$$

where $\mathbf{x}$ and $\mathbf{y}$ are complex vector indicating transmitted and received signal respectively, $\mathbf{n}$ is a noise vector and finally, $\mathbf{H}$ is a channel-information matrix. CSI is collected for each subcarrier and the value for each of them is defined as a complex number with a real component ($H_r^{(i)}$) and an imaginary component ($H_{im}^{(i)}$). With that, denoting the subcarrier index as i , we can calculate amplitude from raw CSI,

$$A^{(i)} = \sqrt{(H_{im}^{(i)})^2 + (H_r^{(i)})^2} \quad (2)$$

, and phase as well,

$$\phi^i = \text{atan2}(H_{im}^{(i)}, H_r^{(i)}) \quad (3)$$

B. Related Work

TABLE I
COMPARISON BETWEEN CSI COLLECTING HARDWARE.

	USRP	Atheros	Intel 5300	ESP32
Cost	\$8,400	\$11	\$11	\$6
Size (cm)	26.7x21.8	2.98x2.82	3x2.68	5.5x2.8
Weight	1.6kg	>1kg (*)	>1kg (*)	10g

(*): Including the computer.

The range of applications in smart homes with WiFi sensing via CSI can be wide; including but not limited to human activity recognition, localization, crowd counting, and occupancy detection. Thereby, many interesting works surrounding the topic have emerged. The survey in [9] did fantastic work covering the many techniques and existing systems that operate on the premise of WiFi sensing. The ESP32 microcontroller is a popular hardware choice for CSI collection due to its low cost, small size, and capability. As shown in Table I, the price for the ESP32 (WROOM32 model) is the lowest in comparison with the Ettus Research USRP X300, the Atheros AR9580 NIC, and the Intel WiFi Link 5300 wireless NIC. While the Intel 5300 and Atheros NICs are physically smaller than the ESP32, they must be connected to a computer to function. This requirement adds significant weight to the overall setup, making them less convenient than the ESP32, which operates as a standalone device.

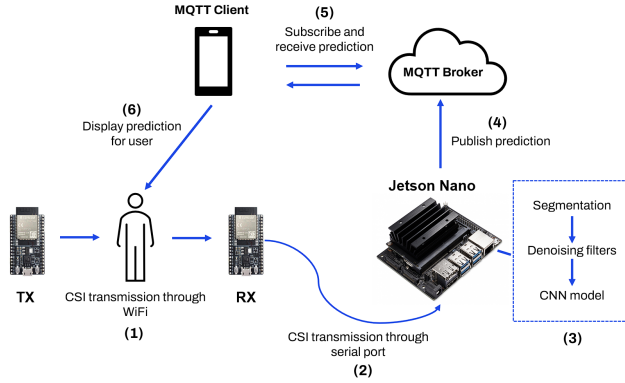


Fig. 1. Processing flow of the proposed system.

Accordingly, this paper will focus on WiFi sensing using the ESP32 device.

For occupancy detection, the authors in [10] were able to monitor human presence through the wall. In a setup of one person moving through the hallway, a result of both the true positive and true negative rate reaching 1.0 was achieved. Not only that, the direction of movement was also accurately predicted by applying the same binary algorithm for human detection.

For localization and crowd counting, H. Choi et al. [11] constructed a system performing both simultaneously using four pairs of ESP32 nodes as WiFi links. They treated the localization and crowd-counting tasks as classification and regression analysis respectively. Doing so in 2 different rooms, they achieved a counting error of 0.35 median absolute error (MAE) and 91.4% of localization accuracy with five people in a small-sized meeting room and 0.41 MAE and 98.1% of localization accuracy with 10 people in a medium-sized seminar room.

Similar to our focus, [12] explored human activity recognition which achieved varied accuracies across four activity sets: wrist movements (85.2% overall), finger movements (97.1%), whole-body movements (80.7%), and exercise equipment use (96.9%). Their lower accuracy with whole-body movements was due to confusion between the left and right limbs. We address this limitation in our work through increased transmitter-receiver distance. While their attempt to identify individuals by equipment use was unsuccessful, our model focuses on accurately recognizing the activity itself.

As the trend of using ESP32 is still recent, the number of research covering this is fairly few, and even a smaller number for human activity recognition. The goal of this paper is to learn further about this aspect of WiFi sensing applications and try to implement a complete system.

III. PROPOSED SYSTEM

In this section, we introduce the components of our WiFi sensing system. The system includes:

- A pair of ESP32 microcontrollers that act as a transmitter (TX) and receiver (RX) for CSI collection.
- The Jetson Nano is used for edge inference.
- An MQTT broker (server) hosted on the cloud.
- An Android application receiving information.

An overview of the processing flow is shown in Fig. 1. It contains the following steps:

- 1) CSI is transmitted via WiFi signals from the TX to the RX. During this, an activity will be performed in the middle of the transmission path.
- 2) The RX will then forward the received CSI data to the nearby Jetson Nano through a serial port.
- 3) A series of data processing steps will be run by the Jetson Nano. After this, a prediction will be made by the machine learning model.
- 4) The Jetson Nano will then connect to the MQTT broker and publish its prediction to a predefined topic.
- 5) The MQTT client subscribing to that topic will receive the predictions from the broker.
- 6) Finally, the prediction will be displayed on the application interface for the end user to see.

Next, we shall elaborate more on these components in the following parts.

A. CSI Collection

All CSI collection processes are done on the ESP32 microcontrollers and the ESP32-CSI Toolkit [4]. The toolkit provides a framework to configure the ESP32 nodes to operate in access point mode or station mode with many other options to work with, such as rate of transmission. We choose a sampling rate of 100Hz

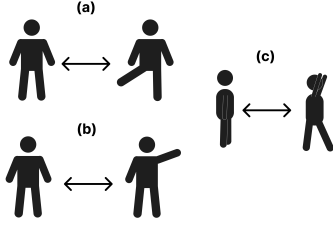


Fig. 2. Activities for classification: (a) Raising left leg. (b) Raising right arm. (c) Stretching out.

for the transmission from the TX to RX, which means approximately 100 CSI packets are collected for each second.

We collected CSI data for the following three activities: (i) raising right arm (RA), (ii) raising left leg (RL), and (iii) stretching out (SO). An illustration of how these activities are performed can be seen in Fig. 2.

B. Data Pre-processing

1) *Segmentation*: As mentioned in Section III-A, the RX receives about 100 CSI packets per second and since the duration of each activity is roughly 1-2 seconds, we decided to split the data stream into 200-packet segments for the time window of 2 seconds. The system will calculate the amplitude values of each CSI segment using the mathematical expression stated in Section II-A.

2) *Data Denoising*: Raw CSI can be significantly noisy which affects the performance of machine learning models. In order to smoothen the data, two filters are applied which are the Hampel filter and the Savitzky-Golay filter. The first filter is used to remove outliers without changing the rest while the latter is used for reducing the overall noise while maintaining the shape of the waveform.

Fig. 3 shows the amplitude values of one CSI segment for each activity after pre-processing, it is noted that there is a distinct pattern between the three activities.

C. Machine Learning Models

After pre-processing, we begin to select several machine learning models for the classification task. In this work, we use a support vector classifier (SVC), a Logistic Regression classifier (LRC), and a Random Forest Classifier (RFC). Furthermore, we build a deep learning classifier model that uses the convolutional neural networks (CNN) architecture. As pointed out by another survey in [13], the CNN model excels at extracting spatial features from CSI data which leads us to choose this specific model architecture. The purpose of

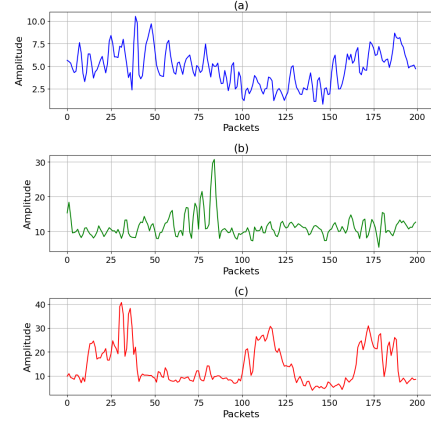


Fig. 3. Amplitude values of one subcarrier's CSI for different activities: (a) Raising left leg. (b) Raising right arm. (c) Stretching out.

training multiple models is to compare the performances of these models and then choose the most suitable one for edge deployment.

D. MQTT Broker

The MQTT protocol provides lightweight and secure means of communication between interconnected devices and thus, it has been used in numerous IoT applications [14]. Using the HiveMQ Cloud platform [15], we create a cluster that includes an MQTT broker running on a shared virtual machine. For client devices to establish connections to the broker, we create two accounts, one for the Jetson Nano and one for the Android application. Each account contains essential information such as username, password, and ClientID—a unique identifier that distinguishes each MQTT client.

E. Android Application

Our mobile Android application, developed in Android Studio using Java, allows users to subscribe to an MQTT server and view predicted activity information within a dedicated user interface fragment. Upon successful login, users are directed to the home interface. This interface displays the following information: Activity name, activity count (number of times performed), and most recent activity timestamp.

IV. EVALUATION

A. Dataset creation

For management purposes, we define one minute of raw CSI collection as one session. With that, each activity is performed for a total of six sessions. Only one volunteer performed all three activities during the data collection process. After segmentation, we have a

dataset of 1800 samples for each activity. We then split this dataset into a ratio of 6:2:2 for training, validation, and testing.

All sessions were collected in an indoor environment with the volunteer standing in the line of sight of the TX/RX pair. The TX and RX were placed 3 meters apart from each other.

B. Model Performance

TABLE II
PERFORMANCE COMPARISON BETWEEN LEARNING MODELS.

	LRC	SVC	RFC	CNN
Accuracy	0.82	0.83	0.93	0.95
Precision	0.82	0.84	0.93	0.94
Recall	0.82	0.83	0.93	0.94

For conventional machine learning models, an additional step of data flattening was needed. As these models often expect one-dimensional data, our dataset where each sample is a two-dimensional matrix would not be suitable without flattening. After that, we can begin training and testing the models:

- LRC: Trained with L2 regularization, C value of 1.0, and the liblinear solver for parameter optimization. The training time was 8 seconds.
- SVC: Trained with a linear kernel and C value of 1.0. The training time was 10 seconds.
- RFC: Trained with 100 decision trees, each of which had a maximum depth value of 20. The training time was approximately 1 second.

The result is listed in Table I where the RFC model achieved the highest accuracy, exceeding SVC and LRC marginally. Notably, the RFC model also leads in precision and recall. Yet, as mentioned, data flattening before training may compromise spatial patterns, potentially affecting model reliability [16].

Using the CNN model will allow us to avoid the previously mentioned problem by preserving the data structure and capturing spatial features in CSI data. We train our CNN classifier model using the architecture as follows: input layer, convoluted layer with a kernel size of 7x7, max pooling layer, convoluted layer with a kernel size of 5x5, flattening layer, two dense layers with a dropout layer in-between, and an output layer. The Adam optimization algorithm, a popular extension of Stochastic Gradient Descent, is used to optimize the cross-entropy loss function during training.

After training the CNN model for 43 epochs (early stopping with the patience of 5) and a batch size of 128, the validation accuracy and loss of the model were

91.9% and 2.2% respectively. Next, we have the model make its predictions for the testing set. The classification result can be seen in the confusion matrix in Fig. 4 where an overall accuracy of 95.6% was achieved by the model. To be more specific, for LL, RA, and SO, the model was able to classify with 96.6%, 94.2%, and 95.8% in terms of accuracy respectively. To assess both the model's precision and recall values, we next calculate the F1-score values which consider both false positives and false negatives. The result of each class is as follows: LL - 94.1%, RA - 97.0%, SO - 95.8%. All of them are higher than 90.0% showing a good balance between precision and recall.

Although the CNN model only outperformed the RFC model by a small margin, its ability to preserve and learn from the complex structure of CSI data without an additional preprocessing step is ideal for our system. Thus, we chose CNN as our model for edge deployment on the Jetson Nano as output with less latency will be delivered due to shorter preprocessing time.

C. System Deployment

As we trained the model using the TensorFlow library, the exported model will be in a SavedModel format. Format conversion is needed for the model to deliver better performance on the Jetson Nano. First, we convert the model to an Open Neural Network Exchange (ONNX) format using the tf2onnx library. With the model in ONNX format, we continue to convert it to the TensorRT format. This two-step conversion process is recommended due to its generally low memory consumption while delivering high inference throughput [17]. On the Jetson Nano, during the initial one-minute session, the model had a throughput of 217.871 queries per second (qps) and an average latency of 4.5 milliseconds. From these figures, we can calculate the inference rate of the model to be approximately 48.42Hz. Further experimenting for multiple sessions shows similar results with small variance.

As we segmented each activity into two-second frames, the model was set to push its prediction to the MQTT broker every two seconds, introducing a slight delay between the model, the broker, and the app. While specific end-to-end latency measurements weren't conducted due to the highly scenario-dependent nature of message delivery time, we observed an overall response time suitable for real-world scenarios. This suggests that the efficient nature of MQTT and small payloads contribute to a negligible delay that does not significantly impact the system's performance.

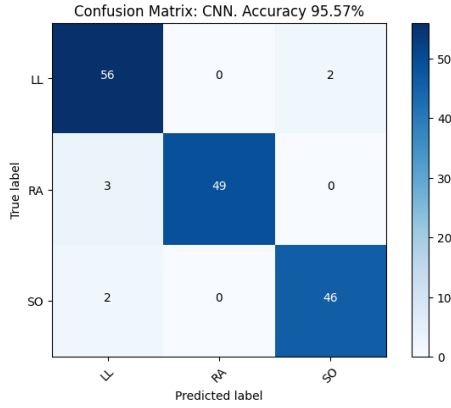


Fig. 4. Confusion matrix and overall accuracy of the CNN model.

V. CONCLUSION

In this paper, we proposed a fully functional WiFi sensing system for human activity recognition through the extraction of CSI signals. We highlighted the limitations of existing solutions due to privacy concerns, cost, and hardware requirements. With that, we presented our system including a pair of ESP32 microcontrollers, a Jetson Nano, MQTT server communication, and an Android application. The ESP32's CSI Toolkit efficiently captured and processed CSI data, demonstrating its potential as an affordable platform for WiFi sensing. Furthermore, the evaluation of various machine learning and deep learning models identified suitable options for efficient activity classification on the Jetson Nano edge device. Using the proposed system, we collected and evaluated CSI signal data for three activities: (i) raising left hand, (ii) raising right hand, and (iii) stretching out. As demonstrated, through edge computing and the use of MQTT protocol, all three activities can be recognized at high accuracy and low latency by our system. In future work, we will consider upscaling the number of activities as well as building deep learning models that are independent. For more practicality, we will also work on integrating WiFi sensing into camera systems.

ACKNOWLEDGMENT

This research is funded by the University of Information Technology - Vietnam National University HoChiMinh City under grant number D1-2024-13.

REFERENCES

- [1] Piotr Grzesik and Dariusz Mrozek. Combining machine learning and edge computing: Opportunities, challenges, platforms, frameworks, and use cases. *Electronics*, 13(3), 2024.
- [2] Xiaoming Wang, Hongliu Yu, Søren Kold, Ole Rahbek, and Shaoping Bai. Wearable sensors for activity monitoring and motion control: A review. *Biomimetic Intelligence and Robotics*, 3(1):100089, 2023.

- [3] Isura Nirmal, Abdelwahed Khamis, Mahbub Hassan, Wen Hu, and Xiaoqing Zhu. Deep learning for radio-based human sensing: Recent advances and future directions. *IEEE Communications Surveys & Tutorials*, 23(2):995–1019, 2021.
- [4] Steven M. Hernandez and Eyuphan Bulut. Lightweight and Standalone IoT Based WiFi Sensing for Active Repositioning and Mobility. In *21st International Symposium on "A World of Wireless, Mobile and Multimedia Networks" (WoWMoM) (WoWMoM 2020)*, Cork, Ireland, June 2020.
- [5] Minh Tuan Pham, Long Thai Hoang, Ha Dang Tran Hong, Tien Do Minh, and T H Phuoc Nguyen. An WiFi CSI signal enhancement framework for activity recognition using machine learning automatic segmentation. *SN Computer Science*, 5(5):524, May 2024.
- [6] Daniel Halperin, Wenjun Hu, Anmol Sheth, and David Wetherall. Tool release: Gathering 802.11n traces with channel state information. *ACM SIGCOMM CCR*, 41(1):53, Jan. 2011.
- [7] Yaxiong Xie, Zhenjiang Li, and Mo Li. Precise power delay profiling with commodity wifi. In *Proceedings of the 21st Annual International Conference on Mobile Computing and Networking*, MobiCom '15, page 53–64, New York, NY, USA, 2015. ACM.
- [8] Umer Saeed, Syed Yaseen Shah, Syed Aziz Shah, Haipeng Liu, Abdullah Alhumaidi Alotaibi, Turke Althobaiti, Naem Ramzan, Sana Ullah Jan, Jawad Ahmad, and Qammer H. Abbasi. Multiple participants i& rsquo; discrete activity recognition in a well-controlled environment using universal software radio peripheral wireless sensing. *Sensors*, 22(3), 2022.
- [9] Steven M. Hernandez and Eyuphan Bulut. WiFi Sensing on the Edge: Signal Processing Techniques and Challenges for Real-World Systems. *IEEE Communications Surveys and Tutorials*, 2022.
- [10] Steven M. Hernandez and Eyuphan Bulut. Adversarial Occupancy Monitoring using One-Sided Through-Wall WiFi Sensing. In *2021 IEEE International Conference on Communications (ICC): IoT and Sensor Networks Symposium (IEEE ICC'21 - IoTSN Symposium)*, Montreal, Canada, June 2021.
- [11] Hyuckjin Choi, Manato Fujimoto, Tomokazu Matsui, Shinya Misaki, and Keiichi Yasumoto. Wi-cal: Wifi sensing and machine learning based device-free crowd counting and localization. *IEEE Access*, 10:24395–24410, 2022.
- [12] Steven M. Hernandez, Md Touhiduzzaman, Peter E. Pidcoe, and Eyuphan Bulut. Wi-PT: Wireless Sensing based Low-cost Physical Rehabilitation Tracking. In *2022 IEEE International Conference on E-health Networking, Application & Services (IEEE Healthcom 22)*, Genoa, Italy, October 2022.
- [13] Chenning Li, Zhichao Cao, and Yunhao Liu. Deep ai enabled ubiquitous wireless sensing: A survey. *ACM Computing Surveys*, 54:1–35, 03 2021.
- [14] Biswajeeban Mishra and Attila Kertesz. The use of mqtt in m2m and iot systems: A survey. *IEEE Access*, 8:201071–201086, 2020.
- [15] HiveMQ Team. *HiveMQ Documentation*. HiveMQ. Accessed on November 15, 2023.
- [16] Yong Zhang, Yuqing Yin, Yujie Wang, Jiaqiu Ai, and Dingchao Wu. Csi-based location-independent human activity recognition with parallel convolutional networks. *Computer Communications*, 197:87–95, 2023.
- [17] Mari Carmen Bañuls. Tensor network algorithms: A route map. *Annual Review of Condensed Matter Physics*, 14(1):173–191, March 2023.